

Verifiable Observability: Project Overview and Architecture

1. Executive Summary

The Problem

As AI agents become more autonomous, they are being deployed in high-stakes domains like **Finance**, **Healthcare**, and **Code Execution**. However, LLMs (Large Language Models) are inherently probabilistic "black boxes." When an AI agent executes a \$50,000 trade, accesses a patient's medical record, or runs a shell command on a server, organizations need a way to **mathematically prove** that the agent followed Standard Operating Procedures (SOPs).

The Solution

Verifiable Observability is a runtime monitoring and enforcement framework for AI agents. It does not just let the agent run wild; it actively wraps the agent in a verification layer. It observes the agent's reasoning, grades its compliance against predefined rules, and uses a hard constraint system to block unsafe actions before they happen.

2. Core Architecture (How it Works)

The system is built on a "Turn-Based Agent Loop." Before any tool or action is executed by the AI, it must pass through our safety architecture:

1. **Strategy Profiler:** When a task is received (e.g., "*Prescribe amoxicillin*"), the Profiler analyzes the text, assigns it a **Task Type** (e.g., `medication_management`), assigns a **Risk Tier** (LOW, MEDIUM, HIGH), and loads the correct rules.
2. **Rule Bank:** The database of Standard Operating Procedures. It acts as the "ground truth" for what the agent *should* do. For example: "*If prescribing medication, the agent MUST first check patient allergies.*"
3. **Constraint Compliance Monitor (CCM):** The strict enforcement firewall. While the Rule Bank grades the agent's behavior, the CCM actively intercepts API calls. It can output three decisions:
 - **ALLOW:** Safe to proceed.
 - **FLAG:** Suspicious or requires human review (e.g., a transfer of \$15,000).
 - **BLOCK:** Hard policy violation (e.g., deleting a database or prescribing a controlled substance without a co-sign). The action is stopped immediately.
4. **Metrics Engine (RCR & CCR):**
 - **Rule Compliance Rate (RCR):** Measures how well the agent's reasoning aligned with the expected SOP (0.0 to 1.0).
 - **Constraint Compliance Rate (CCR):** Measures whether the agent triggered any CCM violations (0.0 to 1.0).
5. **Trajectory Store (SQLite Database):** Every thought, action, rule match, and metric is

saved permanently. This ensures total auditability if something goes wrong.

3. The Three Domains

The framework is highly modular and currently supports three high-stakes environments:

Finance

- **Capabilities:** Executing trades, transferring funds, balancing portfolios.
- **Constraints:** Hard blocks on transfers > \$50,000 without prior compliance approval. Flags extremely large share quantities.

Healthcare

- **Capabilities:** Medication management, clinical decision support, HIPAA data access.
- **Constraints:** Blocks access to Protected Health Information (PHI) unless a HIPAA audit log is written first. Blocks controlled substance prescriptions without a physician's co-sign.

Code Execution

- **Capabilities:** Code generation, code review, sandbox execution, system shell commands.
 - **Constraints:** Blocks execution of code in the sandbox without prior static analysis. Blocks privileged `sudo` commands without explicit escalation approval.
-

4. Real-World Walkthrough

Let's look at what happens when a user asks the agent to perform a task:

User Prompt: *"Execute a \$75,000 transfer to an external vendor."*

1. **Profiling:** The Strategy Profiler tags this as `high_value_trade` (Risk: HIGH).
 2. **Reasoning:** The LLM outputs its thought process: *"I will execute the transfer immediately."*
 3. **Action Attempt:** The LLM attempts to call the `execute_transfer` tool with \$75,000.
 4. **CCM Interception:** The `FinanceCCM` catches the tool call. It checks its internal constraints and realizes this exceeds the \$50,000 hard limit without prior approval.
 5. **Outcome:** The CCM returns **BLOCK**. The agent's action is stopped. The `CCR` metric drops to 0.0 for that turn, and the trajectory is marked as **BLOCKED** in the database.
-

5. Drift Detection and Behavioral Regimes

Because AI models can suffer from "drift" (where their behavior degrades over time due to system updates or hallucinations), the project includes an advanced **Metrics Engine**.

It uses statistical trend analysis (Ordinary Least Squares) to track the agent's `RCR` over time. If an agent starts skipping safety checks (e.g., no longer checking account balances before

transfers), the RCR trend will slope downward. The engine will automatically flag this as **Behavioral Drift**, alerting engineers that the AI model is no longer safe to operate autonomously.

We test this using **Behavioral Regimes**—scripted scenarios where we simulate an agent acting maliciously (Adversarial Injection) or slowly forgetting rules (Tool Failure Drift) to prove that our safety monitors catch them every time.

Conclusion

Verifiable Observability transforms AI from an unpredictable "black box" into a deterministic, strictly monitored, and highly auditable employee. By mathematically scoring compliance and physically blocking unsafe actions, it ensures that AI can be safely deployed in the world's most critical industries.